WT32-ETH01 DMX Lighting Node

User Manual & Integration Guide

Firmware Version 1.30.00

Project Documentation

21 June 2026

Contents

Quick Start

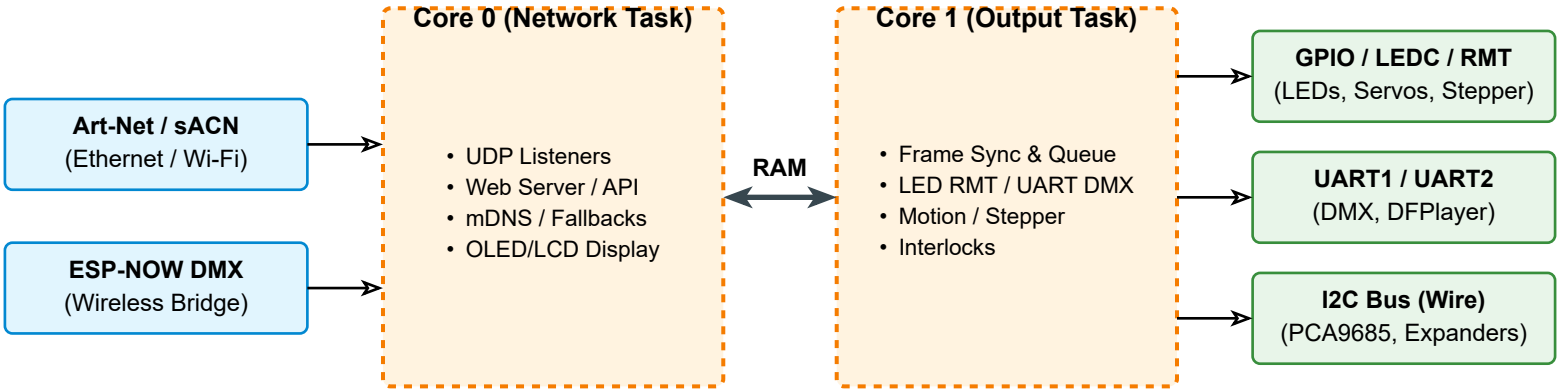
To begin using the WT32-ETH01 DMX Lighting Node, follow these steps:

1. **Power the Node:** Connect a stable 5V power supply to the WT32-ETH01 board. Ensure the supply can deliver sufficient current for the ESP32, Ethernet transceiver, and any connected outputs.
2. **Initial Setup AP:** If Ethernet is disconnected and Wi-Fi client mode is not yet configured, the board automatically starts a configuration Access Point named `ESP32-ArtNet-Setup-XXXX`.
3. **Access Configuration:** Connect your PC or mobile device to the setup AP and navigate to http://192.168.4.1 in a web browser.
4. **Configure Settings:** Navigate through the Network, Outputs, and Protocol tabs to define your system parameters.
5. **Save and Apply:** Click **Save**. The board will validate the settings against hardware constraints and reboot.

Ethernet is strongly recommended for live Art-Net/sACN streaming. Wi-Fi client mode is useful for setup or light-duty/non-critical streaming.

System Architecture

The system architecture utilizes the ESP32's dual-core processor to separate high-priority output timing from network protocol processing. The diagram below illustrates this runtime task split and signal flow.



Network Modes & System Identity

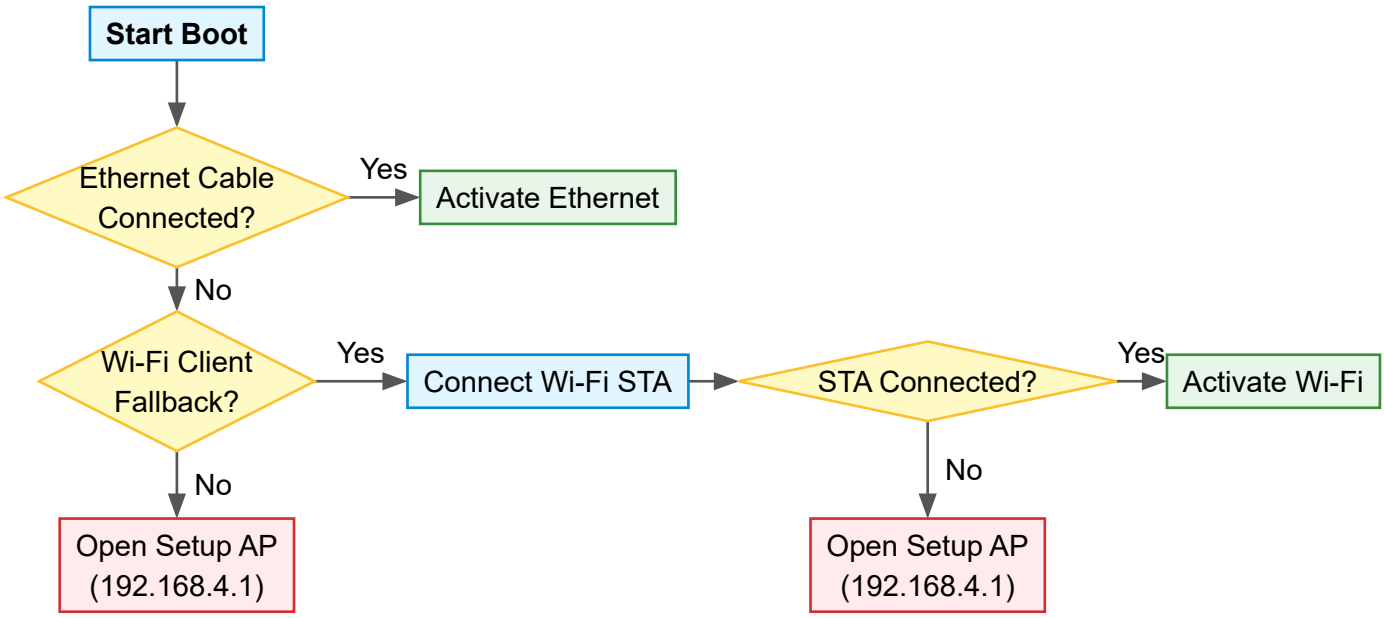
The WT32-ETH01 firmware supports multiple network modes and automated fallbacks to maintain network responsiveness under varying installation conditions.

Network Modes

- **Art-Net Ethernet (Wired):** The main operating mode. The device obtains an IP address via DHCP or static configuration and listens for Art-Net UDP packets on the default port `6454`.
- **ESP-NOW Master:** Receives DMX inputs via Art-Net/sACN and encodes the stream into custom wireless packets sent to remote slave nodes.
- **ESP-NOW Slave:** Receives wireless DMX streams directly from an ESP-NOW Master and drives local output peripherals.

Network Fallback System

During boot in Art-Net Ethernet mode, the device executes a priority-based fallback logic to ensure communication remains accessible if cables are disconnected or signals are weak. The flowchart below outlines this sequence.



System Identity & MAC Suffix

To prevent network clashes when multiple lighting nodes are deployed on the same subnet, the device generates distinct identifiers based on its unique MAC address:

- **Setup AP SSID:** `ESP32-ArtNet-Setup-XXXX` (where `XXXX` is the last 4 hex characters of the MAC address).
- **mDNS Hostname:** `http://[hostname]-[xxxx].local` (where the suffix is lowercase).
- **ArtPollReply Names:** Short Name: `CHAL Node-XXXX`; Long Name incorporates the board type and suffix.

Boot Resiliency & Recovery Mode

If invalid configurations or networking issues cause the firmware to crash or boot loop, the device provides a hardware and software-based **Recovery Mode**:

1. **5-Power Cycle Trigger:** Powering the device on and off consecutively 5 times (rebooting within 15 seconds of boot) forces the board into Recovery Mode. A watchdog timer resets this count to 0 after 15 seconds of stable runtime.
2. **Physical Button Trigger:** Hold the physical **BOOT button (GPIO0)** low during power-up.

When Recovery Mode is activated:

- **Dual Network Access:** The node spins up both Ethernet and an open (no password) Wi-Fi AP named `ESP32-ArtNet-Recovery-XXXX`.
- **mDNS Resolution:** The recovery interface resolves at `http://[hostname]-recovery.local` (with no MAC suffix, simplifying access).
- **Low Power State:** All physical output signals and DMX listening loops are suspended to avoid brownout conditions.

Status LED Blink Codes

The Status LED (default GPIO 5) communicates the system state through distinct repeating flash sequences:

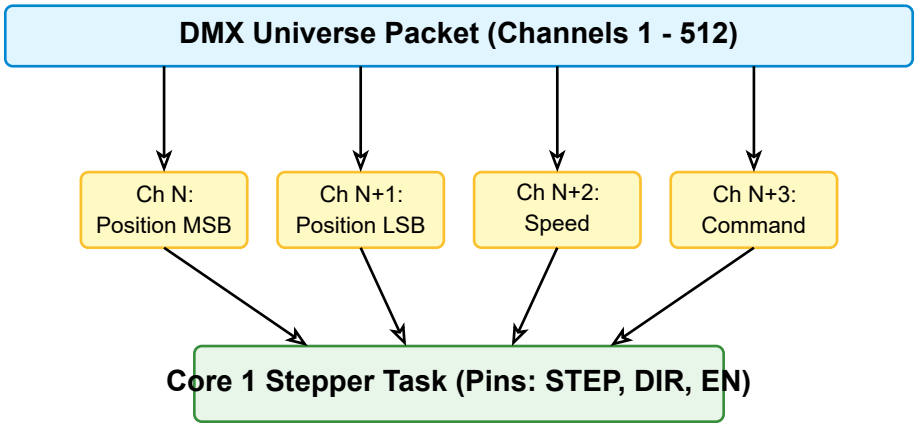
Blinks	Device State
1 Blink	Disconnected or network interface idle
2 Blinks	SoftAP configuration portal active
3 Blinks	Ethernet link established and active
4 Blinks	Wi-Fi STA connection established and active
5 Fast Blinks	ESP-NOW Slave receiving recent valid packets
6 Fast Blinks	Manual Output Test mode active (overriding DMX stream)

Outputs & Capacity Planning

The node allocates outputs dynamically according to a hardware budget scoring algorithm. Output channels map DMX data to physical pins using multiple sources.

Output Concepts

Each output channel is defined by a start universe, start DMX address, type (0–18), source (GPIO, PCA9685, digital expander, I2C DAC), and parameters.



Resource & Compute Scoring System

To guarantee dual-core task stability under heavy configurations, the firmware uses a combined resource and compute score limit. The total budget is capped at **109 points**.

$$\text{Total Score} = \text{Resource Score} + \text{Compute Score} \leq 109$$

The resource score reflects hardware peripherals used, and the compute score reflects CPU ticks based on active output channels and FPS.

Resource Weight Multipliers

Peripheral	Weight	Usage Detail
ESP32 GPIO Pin	0.5	Standard digital input or output pin
LEDC (PWM Channel)	2.5	Direct hardware PWM channel (max 16)
RMT Channel	3.0	RGB LED strips or DMX RMT fallbacks (max 8)
UART Port	8.0	Serial DMX output or DFPlayer MP3 module (max 2)
DAC Channel	2.0	Internal ESP32 DAC (unavailable on WT32-ETH01)
PCA9685 Channel	0.25	I2C-based PWM expander channel
I2C Expander Pin	0.125	MCP23017, TCA9555, or PCF857x expander pin

Output Type Reference

ID	Type Name	Bytes	Compatible Sources	Primary Resource
0	AC Dimmer	1–2	GPIO Only	GPIO 1 + ZC Timer
1	DMX Serial Output	512	GPIO Only (UART/ RMT)	UART (fallback RMT)
2	Relay (On/Off)	1	GPIO, PCA, Ex-pander	GPIO / PCA / Exp 1
3	RGB/RGBW LED Strip	Var	GPIO Only (RMT)	RMT 1 ch
4	Single Color LED	1–2	GPIO, PCA	LEDC 1 / PCA 1
5	Analog RGB/RGBW	3–4	GPIO, PCA (Hybrid)	LEDC 3-4 / PCA 3-4
6	DC Motor (H-Bridge)	1–2	GPIO, PCA (Hybrid)	LEDC 1-3 / PCA 1-3
7	Stepper Motor	3–5	STEP:GPIO; DIR:Any	FastAccelStepper
8	RC Servo	1	GPIO, PCA	LEDC 1 (50Hz) / PCA 1
9	Passive Buzzer	2	GPIO Only	LEDC 1
10	DFPlayer MP3 Module	3	GPIO Only (UART)	UART 1
11	7-Segment (TM1637)	2–4	GPIO Only	GPIO 2
12	7-Segment DD 7-Pin	1–2	GPIO, PCA, Ex-pander	Seg-level routing
13	7-Segment DD 8-Pin	1–2	GPIO, PCA, Ex-pander	Seg-level routing
14	DAC (Analog Out)	1	MCP4725, DAC757x	I2C Bus

15	PWM DAC (RC Filter)	1	GPIO, PCA	LEDC 1 / PCA 1
16	Function Generator	5	GPIO Only	LEDC 1 + timer
17	Solenoid Trigger	1	GPIO, PCA, Expander	GPIO / PCA / Exp 1
18	Smoke Shooter	1	GPIO, PCA, Expander	GPIO / PCA / Exp 2

Pin Interlocks

To maintain hardware safety, configuration rules enforce the following constraints:

- **No GPIO Duplication:** An ESP32 GPIO pin cannot be used by more than one output channel or system setting simultaneously.
- **System Reservation:** GPIO pins configured as Status LED, Zero-Crossing Input, or I2C SDA/SCL are blocked from output selection.
- **PCA9685 Shared Frequency:** All channels on a single PCA9685 chip share one output frequency. Selecting an RC Servo (Type 8) on a PCA9685 chip automatically forces all channels on that chip to 50 Hz.
- **Stepper STEP Pin:** The STEP signal for Type 7 (Stepper Motor) must use a hardware-capable ESP32 GPIO pin due to high frequency toggle requirements. Enable, DIR, and Limit pins can route to digital expanders.
- **Motor EN Pin:** DC Motors operating in Mode 2 (IN1+IN2+EN) require speed modulation on the EN pin. Therefore, the EN pin must route to an ESP32 GPIO or a PCA9685, never a digital expander.

PWM DAC Duty Calibration

For Type 15 (PWM DAC) outputs, you can map the digital 0–255 DMX range to specific hardware duty cycles. This is configured via:

- **Mode:** Custom, 0–10V output converter, or 4–20mA transmitter.
- **Min Duty:** Duty cycle percentage multiplied by 100 (e.g. 500 = 5.0%) mapped to DMX 0.
- **Max Duty:** Duty cycle percentage multiplied by 100 (e.g. 9500 = 95.0%) mapped to DMX 255.

Linear interpolation mapping runs inside Core 1′s update loop to set the LEDC duty register correctly.

ESP-NOW Wireless DMX Bridge

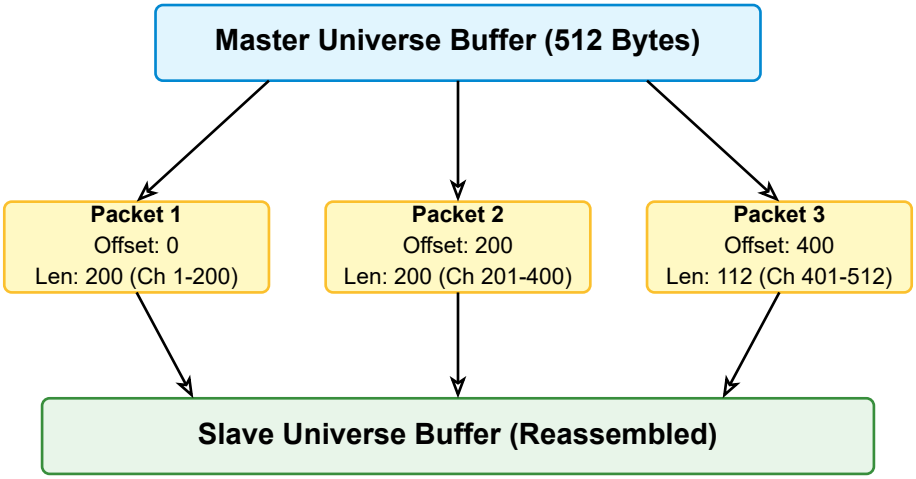
The ESP-NOW Wireless Bridge allows wireless forwarding of DMX universes from a network-connected Master board to one or more remote Slave boards.

Recommended Setup Flow

1. **Configure Slave:** Set the remote board to ESP-NOW Slave mode. Access its Web UI, navigate to the ESP-NOW tab, and copy its physical Wi-Fi MAC Address.
2. **Configure Master:** Set the central network-connected board to ESP-NOW Master mode. Under the ESP-NOW Peer List, click **Add Peer**.
3. **Define Routing Range:** Paste the MAC Address and input the start/end universe and channel ranges mapping to this slave.
4. **Save and Sync:** Save settings. The Master starts encoding and transmitting matching DMX packets.

Universe Chunking & Custom Protocol

ESP-NOW frames have a physical payload limit of 250 bytes. To transmit a full 512-channel DMX universe, the Master splits the universe data into chunks of up to 200 bytes. The custom packet structure uses a 12-byte header followed by DMX data bytes.



Payload Layout

The custom firmware packet header consists of:

1. `Signature` (3 bytes): Ascii `"DMX"`
2. `Universe` (2 bytes): 16-bit unsigned integer (`0..32767`)
3. `Offset` (2 bytes): 16-bit unsigned integer (`0..511`)
4. `Total Length` (2 bytes): 16-bit unsigned integer (`1..512`)
5. `Chunk Length` (2 bytes): 16-bit unsigned integer (`1..200`)
6. `Reserved` (1 byte): Alignment pad (`0x00`)

Network Stability & Planning

To maximize RF efficiency and reliability, design your layouts around these operational constraints:

- **Narrow Peer Ranges:** Map only the specific channels a slave requires. If a slave only controls a single relay at channel 10, define its peer range on the Master as `U0:CH10` to `U0:CH10`. This prevents transmitting unused DMX bytes.
- **Broadcast Fallback:** If no peers are defined on the Master, it falls back to broadcasting packets to `FF:FF:FF:FF:FF:FF`. Use this only for testing, as broadcast packets consume high airtime and increase collision rates in crowded RF environments.
- **Queue Deferral:** Under the hood, the ESP-NOW callback on Core 0 pushes incoming frames to a FreeRTOS queue. Core 1′s output task handles the de-queueing and physical routing. This prevents blocking Core 0′s Wi-Fi radio stack.

Troubleshooting & Integration Guide

This section details standard debug procedures for hardware integrations, power issues, and software setup.

Common Issues

Device Configuration Portal is Unreachable

- **Symptoms:** Unable to access the Web UI at `192.168.4.1` or the device's Ethernet IP.
- **Solutions:**
  1. Ensure Ethernet activity lights are blinking on the WT32-ETH01 RJ45 jack.
  2. Verify that your PC/phone is connected to the SoftAP SSID `ESP32-ArtNet-Setup-XXXX`.
  3. Trigger Recovery Mode by holding down the **BOOT (GPIO0)** pin during boot, then access the recovery interface.

Only the First Part of an LED Strip Lights Up

- **Symptoms:** Pixels at the start of the strip function correctly, but remaining sections stay dark or freeze.
- **Solutions:**
  1. Check that the **LED Count** field in the channel's output configuration matches the number of physically wired pixels.
  2. Verify that your DMX controller/media server is sending all required universes. For example, a strip with 300 RGB LEDs requires 2 consecutive universes (e.g. Universe 0 and Universe 1).

Brownout Detector Resets (Boot Loops)

- **Symptoms:** The board boot loops immediately after powering up, with console prints showing `Brownout detector was triggered`.
- **Solutions:**
  1. **Separate Power Buses:** Do not draw power for high-current loads (LED strips, motors, relays, or solenoids) directly from the WT32-ETH01′s 3.3V or 5V regulator pins.
  2. **Ground Ties:** Run a dedicated external power supply for high-current loads, and ensure its Ground (`GND`) terminal is connected to the WT32-ETH01 Ground pin.
  3. **Capacitive Filtering:** Add a $100\mu\ \mathrm{F}$ to $1000\mu\ \mathrm{F}$ capacitor across the power lines near high-load devices (like servo or stepper drivers) to buffer transient current spikes.

Integration Warnings

- **GPIO 12 Bootstrapping Warning:** GPIO 12 (MTDI) is a bootstrapping pin that controls the internal flash voltage regulator state during boot. If an external peripheral pulls this pin High or Low incorrectly during startup, the ESP32 will fail to read its flash chip and fail to boot. Ensure any circuit connected to GPIO 12 is in a high-impedance (high-Z) state during power-up.
- **GPIO 16 Reservation:** GPIO 16 must not be selected for user outputs under any configuration. It controls the LAN8720 Ethernet transceiver power state. Overriding it will break network connectivity.